

Database Management Systems (DBMS)

Comprehensive Lecture & Reference Notes (From Fundamentals to Normalization)

Author/Lecturer: Ayomi Dassanayake

Subject: Information Technology / Computer Science

1. Introduction & Database Environment

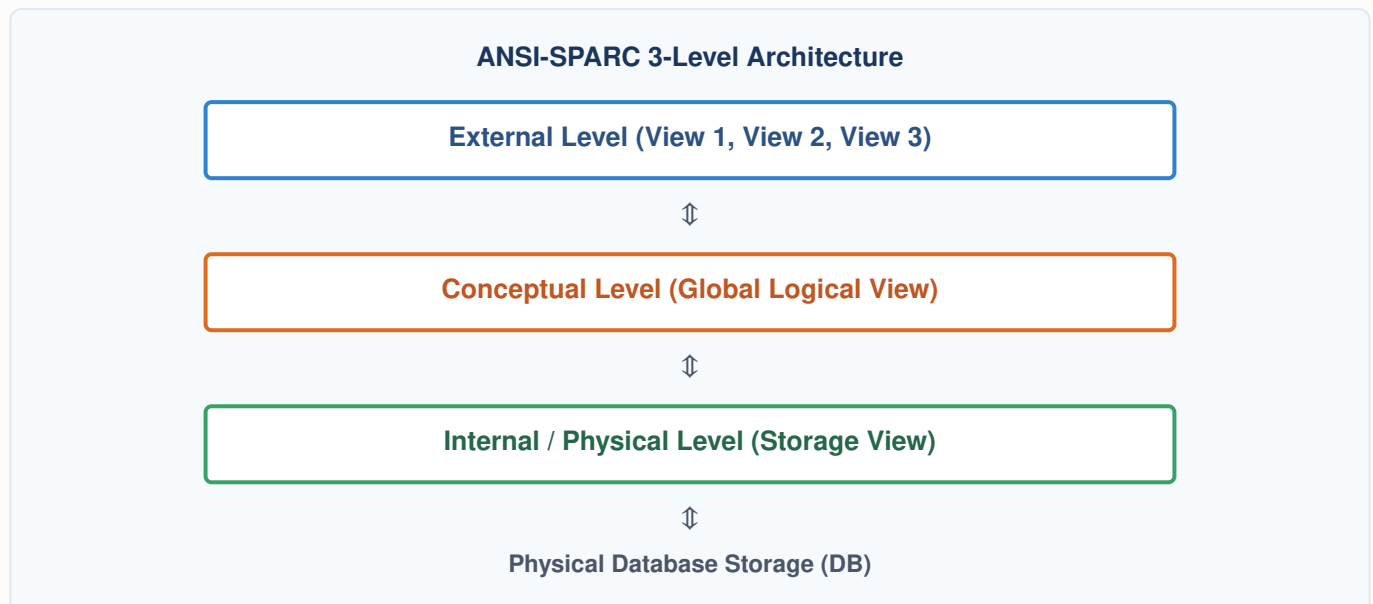
An organization requires systematic information for its day-to-day operations and decision-making processes.

Examples of Organizational Data Requirements:

- **Library:** Book catalogue, borrowing/return records, member details.
 - **School:** Student term exam marks, attendance, registration details.
 - **Bank:** Customer account details, transaction logs, balance statements.
-
- **Data vs. Information:** Data represented in a meaningful and structured context is defined as *Information*.
 - **Multiple Views:** By presenting the same underlying data in different ways, distinct information can be provided to different stakeholders/users based on their needs.
 - **Primary Requirement:** The main objective of an Information System is to store and retrieve data for users in a systematic, efficient, and reliable manner.

2. Three-Level Architecture (ANSI-SPARC Framework)

The ANSI-SPARC 3-level database architecture isolates user applications from the physical database, providing data abstraction and independence.



Detailed Breakdown of Levels:

A. External Level (User Views)

- Represents how individual users view and interact with the database.

- A database can have multiple distinct external views depending on user roles.
- Describes only the portion of the database relevant to a specific user group while hiding irrelevant or sensitive details.
- Allows multiple users to view the same underlying data formatted in different ways simultaneously.

B. Conceptual Level (Logical Level)

- Describes the complete logical structure of the entire database as seen by the Database Administrator (DBA).
- Defines **what** data is stored in the database and the relationships among them.
- Contains entities, attributes, relationships, security constraints, and integrity rules.
- Serves as the central definition from which all external views are derived.

C. Internal / Physical Level

- Describes the physical representation and storage structure of data on secondary storage devices.
- Focuses on **how** data is actually stored (e.g., record formats, file organizations, indexing methods, access paths).
- Managed by the DBMS under the guidance of the Operating System to optimize performance and storage efficiency.

3. Schemas and Mappings

A **Database Schema** is the overall structural description or design of the database. There are three levels of schemas corresponding to the three abstraction levels.

Mappings Between Levels

Mapping refers to the process of transforming requests and responses between different architectural levels, establishing logical connections between entities and their physical storage implementation.

1. External / Conceptual Mapping

Maps a specific External View to the Conceptual Schema. Allows fields/records to have different names, different data types, or combine multiple conceptual fields into a single external view.

2. Conceptual / Internal Mapping

Maps the Conceptual Schema to the Physical Internal Schema. Specifies how conceptual records, fields, and structures are represented physically on disk (e.g., C-style structs, pointers, indexes).

Code Example: Internal Representation of Staff Record

```
struct STAFF {
    int Staff_No;
    int Branch_No;
    char FName[15];
    char LName[15];
    struct date Date_of_Birth;
    float Salary;
    struct STAFF *next;
};
index Staff_No;
index Branch_No;
```

4. Data Independence

Data Independence is the capacity to modify a schema at one level of a database system without altering schema at the next higher level.

Type of Independence	Definition & Scope	Examples / Applications
Logical Data Independence	The capacity to modify the conceptual schema without having to change external schemas or application programs.	Adding new entities/attributes, deleting existing non-essential fields without breaking existing user applications.
Physical Data Independence	The capacity to modify the internal/physical schema without changing the conceptual or logical schema.	• Using new storage devices/hardware • Changing file organizations or data structures • Adding/deleting indexes or access paths

5. Database Development Life Cycle

Developing a database system involves four main structured phases:

1. **Requirements Analysis:** Identifies user needs, business constraints, and system objectives. Defines what the database must accomplish.
2. **Conceptual Design:** Analyzes requirements to create a high-level conceptual schema using models like the Entity-Relationship (ER) Model. This phase is completely DBMS-independent.
3. **Logical Design:** Maps the conceptual schema into a software-dependent logical database model (e.g., Relational Model, Network Model). Defines tables, views, and relationships.
4. **Physical Design:** Translates the logical design into specific storage structures, file formats, and indexing choices within a target DBMS. Focuses heavily on performance metrics like response time, throughput, and space utilization.

6. Entity-Relationship (ER) Modeling

The Entity-Relationship (ER) model describes the data requirements of an information system at the conceptual level.

Core Components & ER Symbols

Component	ER Symbol	Description & Examples
Entity	Rectangle	A real-world object or concept distinguishable from others. • <i>Physical</i>: Person, Car, Book • <i>Abstract</i>: Subject, Course
Attribute	Ellipse	Properties or characteristics of an entity (e.g., <code>Student_ID</code> , <code>Color</code> , <code>Name</code>).
Key Attribute	Underlined Ellipse	An attribute that uniquely identifies an entity instance (e.g., <u><code>SSN</code></u> , <u><code>Student ID</code></u>).
Multi-valued Attribute	Double Ellipse	An attribute that can hold multiple values for a single entity (e.g., <code>Hobbies</code>).
Derived Attribute	Dashed Ellipse	An attribute calculated from another attribute (e.g., <code>Age</code> derived from <code>Birthdate</code>).
Composite Attribute	Connected Hierarchy	An attribute composed of sub-attributes (e.g., <code>Name</code> split into <code>First</code> & <code>Last</code>).
Relationship	Diamond	An association among two or more entities (e.g., <code>Student Enrolls In Subject</code>).

Cardinality Ratios

Defines the maximum number of entity instances that can participate in a relationship set:

- **One-to-One (1:1)**: An instance in Entity A is associated with at most one instance in Entity B.
- **One-to-Many (1:M)**: An instance in Entity A is associated with many instances in Entity B.
- **Many-to-One (M:1)**: Multiple instances in Entity A map to a single instance in Entity B.
- **Many-to-Many (M:N)**: Multiple instances in Entity A map to multiple instances in Entity B (e.g., `Employee Works_On Project`).

Types of Relationships

- **Binary Relationship**: Connects two entity types (e.g., `Employee` — `WORKS_ON` — `Project`).
- **Ternary Relationship**: Connects three entity types simultaneously (e.g., `Course`, `Classroom`, and `Day` meeting together).

7. Keys in Relational Model

Keys are critical for uniquely identifying rows (tuples) in a relational table and maintaining relationships across tables.

- **Super Key**: An attribute or set of attributes that uniquely identifies a row in a relation.
- **Candidate Key**: A minimal Super Key containing no redundant attributes. A relation can have multiple candidate keys.
- **Primary Key**: The specific Candidate Key chosen by the database designer to uniquely identify tuples in a table.

- **Alternate Key:** Candidate keys that were not selected as the Primary Key.
- **Composite Key:** A key composed of two or more attributes combined to ensure uniqueness.
- **Foreign Key:** An attribute in one table that references the Primary Key of another table, establishing a relational link.

8. ER Diagram to Relational Schema Mapping

1:1 Relationship Mapping

Can be implemented using one of three strategies depending on mandatory/optional participation:

- *Method I (Single Table):* Staff(emp_no, name, cont_no, start, end, position, salary)
- *Method II (Foreign Key in Primary Entity):* Staff(emp_no, name, cont_no) & Contract(cont_no, start, end, position, salary)
- *Method III (Foreign Key in Secondary Entity):* Staff(emp_no, name) & Contract(cont_no, start, end, position, salary, emp_no)

1:M Relationship Mapping

Place the Primary Key from the '1' side into the 'M' side table as a Foreign Key:

- Course(course_no, c_name)
- Student(matric_no, st_name, dob, course_no)

M:N Relationship Mapping

Create a new intermediate/junction table containing the Primary Keys of both participating entities as Foreign Keys:

- Student(matric_no, st_name, dob)
- Module(module_no, m_name, level, credits)
- Studies(matric_no, module_no) (*Composite Primary Key: matric_no + module_no*)

9. Functional Dependencies (FDs)

A **Functional Dependency** $X \rightarrow Y$ specifies that the value of attribute set X uniquely determines the value of attribute set Y .

Examples of Functional Dependencies:

- EmployeeNumber $\rightarrow$ FirstName, LastName, Department, Email
- (Weight, Shape) $\rightarrow$ ObjectColor
- SKU $\rightarrow$ SKU_Description, Department, Buyer
- Buyer $\rightarrow$ Department

10. Relational Normalization

Normalization is the systematic process of organizing data in a relational database to minimize data redundancy and prevent insertion, update, and deletion anomalies by decomposing unnormalized relations into smaller, well-structured relations.

Normal Forms Overview

1. Unnormalized Form (0NF) to First Normal Form (1NF)

- **1NF Requirement:** All attributes must contain atomic (indivisible) values, and there must be no repeating groups or nested relations.
- *Action:* Eliminate multi-valued attributes and composite attributes by expanding rows or creating separate tables with primary keys propagated.

2. Second Normal Form (2NF)

- **2NF Requirement:** Must be in 1NF and must have **no partial dependencies** (no non-key attribute is dependent on only a part of a composite primary key).
- *Action:* Decompose tables so that non-key attributes depend on the entire primary key.

3. Third Normal Form (3NF)

- **3NF Requirement:** Must be in 2NF and must have **no transitive dependencies** (no non-key attribute depends on another non-key attribute).
- *Action:* If $X \rightarrow Y$ and $Y \rightarrow Z$, separate $Y \rightarrow Z$ into a new table.

Summary Checklist for Database Normalization:

- **1NF:** Atomic values only, no repeating groups.
- **2NF:** 1NF + Full functional dependency on primary key (no partial key dependency).
- **3NF:** 2NF + No transitive non-key dependencies.